

Back End Javascript

CSC 317 - Fall 2025

Back-End JavaScript Scope 🔭

- Runs on the server side, via Node.js.
- Manages state:
 - Database connections
 - API calls
 - Business Logic
- Provides Routing (endpoints)
- Middleware, Proxying

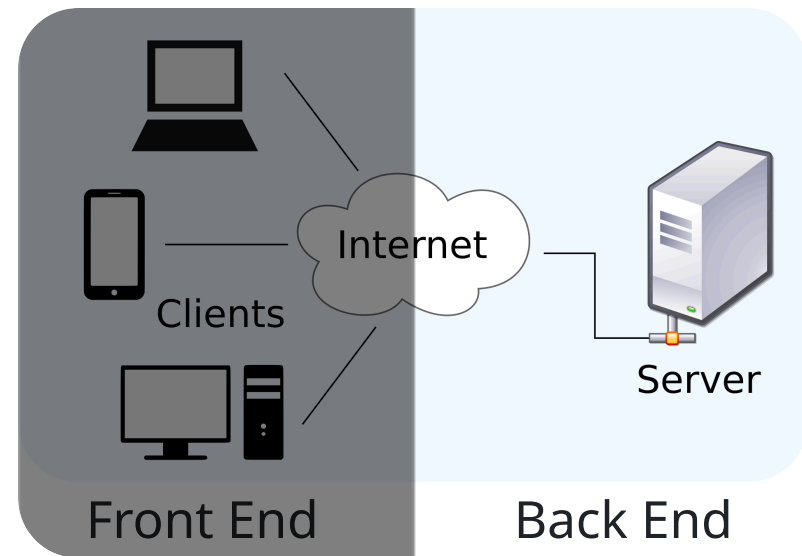


Image Source: [Wikipedia](#)

Overview:

- Part 1 - Node.js 
- Part 2 - Express 
- Part 3 - Databases 
- Part 4 - External APIs 

Part 1 - Node.js

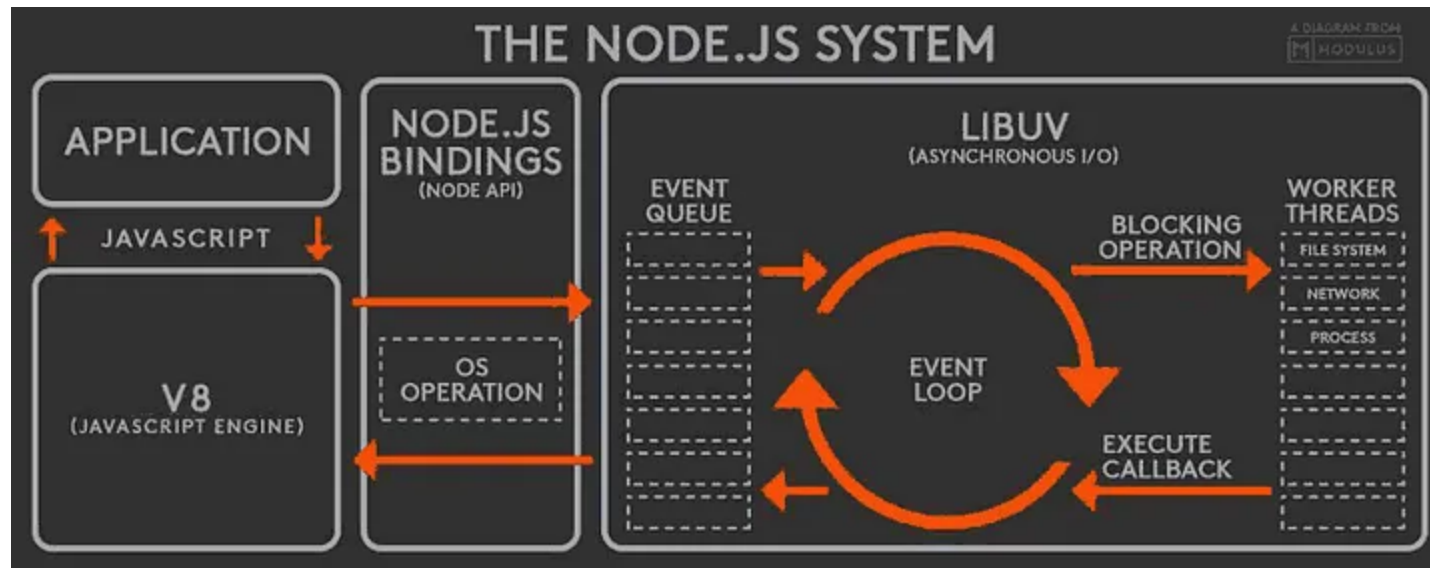
- Runtime
- Event Loop
- Installing Node
- Running Node
- Modules and Exports

Node.js Runtime

- Event-driven JavaScript runtime built on the V8 engine
- Single-threaded core that delegates I/O to libuv for async callbacks
- Optimized for real-time apps that need high throughput and low latency
- Lightweight core encourages composing npm packages for features

Event Loop

- Initializes runtime, loads your script, then spins the event loop
- Phases execute queued callbacks FIFO until their list empties
- Kernel signals finished I/O; callbacks land in the poll queue
- Loop exits cleanly when timers and pending async work are drained



[image source](#)

Event Loop Phases

- timers → run `setTimeout` and `setInterval` callbacks after thresholds
- pending callbacks → handles residual I/O callbacks except close events
- poll → retrieves new I/O, executes ready callbacks, can block waiting
- check → runs `setImmediate` callbacks queued by your code
- close callbacks → fires clean-up handlers such as `socket.on('close')`

Installing node with nvm (Recommended)

- Install nvm (Node Version Manager) once per machine to keep node/npm isolated per user
- Use nvm to install node. npm (Node Package Manager) will be installed too
- Grab the install script from GitHub, then source your shell init file
- Use `nvm install --lts` (or a specific version) to download Node and npm
- `nvm use` switches your active version without touching system packages

```
$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.2/install.sh | bash
$ source ~/.bashrc
$ nvm install --lts
Downloading and installing node v22.x...
$ nvm use --lts
Now using node v22.x (npm v10.x)
```


Running Node Locally

- `node` starts the REPL (Read-Eval-Print Loop) for quick experiments
- `node file.js` executes a script once; combine with shebang for CLIs
- `npm` ships with Node to install packages locally or globally (`-g`)
- `nvm` manages multiple Node versions per user shell

```
$ node
> const greeting = 'Hello from the REPL';
> greeting.toUpperCase();
'HELLO FROM THE REPL'
> .exit
```

```
$ cat hello.js
console.log('Hello from a script');
```

```
$ node hello.js
Hello from a script
```

Initializing a Node Project 🚀

- Run `npm init` in an empty directory to generate the first `package.json`
- Interactive prompts capture name, entry point, scripts, and license
- `npm init -y` accepts defaults for fast prototyping or demos

```
$ mkdir my-app && cd my-app  
$ npm init -y  
Wrote to /path/to/my-app/package.json
```

package.json Essentials

- Holds package metadata plus `scripts` , `dependencies` , and `devDependencies`
- `main` (or `type: module`) shapes module resolution for CommonJS vs ESM
- Custom scripts (e.g., `start` , `test`) let you alias Node/CLI commands
- Semver ranges document compatible dependency versions for collaborators

Installing Dependencies with npm

- `npm install <pkg>` adds the package to `dependencies` and downloads files
- `npm install --save-dev <pkg>` tracks tooling needed only during development
- Re-running `npm install` hydrates `node_modules` based on `package.json`
- The `package-lock.json` file pins exact versions for reproducible installs

```
$ npm install express
added 50 packages in 1s
```

```
$ tree -L 2
my-app/
|-- node_modules/
|   |-- .bin/
|   `-- express/
|-- package-lock.json
`-- package.json
```

Modules and Exports

- Any file loadable by `require()` is a module; packages add `package.json`
- `module.exports` (or `exports`) defines the public API returned by `require`
- Node caches modules, so repeated `require()` calls reuse the same instance
- `require.main === module` detects direct execution (similar to Python `__main__`)

CommonJS Module Pattern

```
// math.js
function add(a, b) {
  return a + b;
}

module.exports = {
  add,
};
```

```
// app.js
const math = require('./math');

console.log(math.add(2, 3));
```

```
$ node app.js
5
```

When to Reach for Node.js

- Network APIs handling many concurrent connections with minimal overhead
- Real-time services: chat, notifications, pub/sub pipelines
- Streaming data processors and lightweight proxies
- Command-line tooling where fast startup and shared JavaScript code matters

Part 2 - Express

- Express Library
- Express Generator
- Template engines (EJS, PUG, Handlebars)

Express Overview ⚡

- Minimal, unopinionated web framework built atop Node.js
- Compose routing, middleware, and view layers as needed
- Large ecosystem of third-party middleware available via npm
- Foundation of popular stacks (MERN: MongoDB, Express, React, Node)

Installing Express

- Install locally for each project so dependencies live in `node_modules`
- Add the generator globally when you want quick scaffolding
- Commands assume you already ran `npm init`

```
$ npm install express  
$ npm install --save-dev nodemon  
$ npm install -g express-generator  
$ npx express-generator my-app
```

Minimal Express App

- Create `index.js`, import Express, configure a route, listen on a port
- Prefer environment variable fallbacks for deployment
- Use `npm run dev` with `nodemon` for live reload during development

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send('Hello Express!');
});

app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});
```

Middleware Ordering Matters

- Middleware runs in the sequence it is registered
- Routes placed before middleware will skip those later middlewares
- Use scoped middlewares (`app.use('/path', handler)`) to target segments
- Always call `next()` unless you end the response

```
app.use((req, res, next) => {  
  console.log('func1');  
  next();  
});  
  
app.get('/bad', (req, res) => {  
  res.send('Only func1 ran before this route');  
});  
  
app.use((req, res, next) => {  
  console.log('func2');  
  next();  
});
```

Scoped Middleware Chain

- Mounting middleware on a path ensures it fires before matching routes
- Great for authentication, logging, or feature-specific setup

```
app.use('/good', (req, res, next) => {  
  console.log('good1');  
  next();  
});  
  
app.use('/good', (req, res, next) => {  
  console.log('good2');  
  next();  
});  
  
app.get('/good', (req, res) => {  
  res.send('good1 → good2 ran first');  
});
```

Routing Basics

- Express matches routes with `app.METHOD(path, handler)`
- Method names align with HTTP verbs: `get` , `post` , `put` , `delete` , etc.
- `req` , `res` , and `next` object references power each handler
- Combine with middleware to parse JSON payloads or serve static assets

```
app.use(express.json());

app.post('/login', (req, res) => {
  const { username } = req.body;
  res.send(`Logged in ${username}`);
});
```

Route Parameters & Patterns

- Dynamic parameters live in `req.params`
- Pattern tokens (`?` , `+` , `*`) match optional segments
- `req.query` reads query string pairs

```
app.get('/user/info/:id', (req, res) => {  
  res.send(`User info id param: ${req.params.id}`);  
});
```

```
app.get('/ab?cd', (req, res) => {  
  res.send('Matched /acd or /abcd');  
});
```

Request Helpers

- `req.params` → URL parameters, `req.query` → query string
- `req.body` populated after body-parsing middleware
- `req.get(name)` retrieves header values
- `req.protocol`, `req.secure`, `req.hostname`, `req.path` provide context

Response Helpers

- `res.send()` for strings, buffers, or objects (auto `Content-Type`)
- `res.json()` serializes JSON payloads explicitly
- `res.status(code)` sets HTTP status before sending
- `res.redirect()` , `res.render()` , and `res.download()` support common patterns

View Engines with Handlebars

- Configure views directory, engine, and default layout once
- `res.render('viewName', data)` injects data into templates
- Handlebars encourages reusable layouts and partials

```
const path = require('path');
const exphbs = require('express-handlebars');

app.set('views', path.join(__dirname, 'views'));
app.engine('handlebars', exphbs({ defaultLayout: 'main' }));
app.set('view engine', 'handlebars');
```

Error & 404 Handling

- Register a catch-all 404 after all routes to handle unknown paths
- Error-handling middleware must accept four args `(err, req, res, next)`
- Always send a response or call `next(err)` to avoid hanging requests

```
app.use((req, res) => {  
  res.status(404).send('Not Found');  
});  
  
app.use((err, req, res, next) => {  
  console.error('Unexpected error:', err);  
  res.status(500).send('Internal Server Error');  
});
```

Project Structure Tips

- Keep Express apps modular with folders for routes, middleware, models
- Separate reusable helpers (formatters, validators) from route handlers
- Serve static assets from `public/` using `express.static`
- Document scripts and environment variables in `README.md`

```
my-app/  
|-- app.js  
|-- routes/  
|-- middleware/  
|-- controllers/  
|-- views/  
|-- public/  
|-- package.json
```

Modular Routers

- Use `express.Router()` to isolate feature routes and middlewares
- Each router acts like a mini-app mounted under a base path
- Promotes separation of concerns and easier unit testing

Router Example

```
// routes/users.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('List users');
});

router.post('/', (req, res) => {
  res.send('Create user');
});

module.exports = router;
```

```
// app.js
const userRouter = require('./routes/users');
app.use('/users', userRouter);
```

Async Route Error Handling 🧯

- Wrap async handlers in `try { ... } catch (err) { next(err); }`
- Centralized error middleware formats responses consistently
- Utility helpers like `express-async-handler` remove boilerplate

Async Error Example

```
app.get('/data', async (req, res, next) => {  
  try {  
    const payload = await loadData();  
    res.json(payload);  
  } catch (err) {  
    next(err);  
  }  
});  
  
app.use((err, req, res, next) => {  
  console.error(err);  
  res.status(500).json({ message: 'Something went wrong' });  
});
```


Scaffolding with express-generator

- CLI scaffolds controllers, views, static assets, and startup script
- Choose templating (`--view=ejs|pug|hbs`) and CSS preprocessor options
- Start immediately: install deps, run `npm start` , then customize routes

```
$ npx express-generator --view=ejs myapp
$ cd myapp
$ npm install
$ npm start
```

Generator Project Structure

```
myapp/
├─ app.js
├─ bin/
│   └─ www
├─ public/
│   ├── images/
│   ├── javascripts/
│   └─ stylesheets/
├─ routes/
│   ├── index.js
│   └─ users.js
├─ views/
│   ├── error.ejs
│   └─ index.ejs
└─ package.json
```

Key Files in the Scaffold

- `app.js` : wires middleware, routers, view engine, and error handlers
- `bin/www` : boots the HTTP server and reads `PORT`
- `routes/` : feature routers (default `index` + `users`)
- `views/` : templates rendered with your chosen engine
- `public/` : static assets (CSS, client JS, images)

Adding a New Page ("About") +

1. Create a router module under `routes/`
2. Register the router in `app.js` with `app.use(path, router)`
3. Create a view template (EJS, Handlebars, or Pug)
4. Update navigation or layout links if needed

1. Router Example for "About" Page

```
// routes/about.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.render('about', { title: 'About Us' });
});

module.exports = router;
```

2. Register the router in app.js

```
const aboutRouter = require('./routes/about');
app.use('/about', aboutRouter);
```

3. Create "About" Page View (EJS)

```
<h1><%= title %></h1>
<p>Welcome to the About Us page.</p>
```

4. Update your Navigation links if necessary

```
<nav>
  <a href="/">Home</a> | <a href="/about">About</a>
</nav>
```

Template Engine Options

- **EJS** → HTML with `<% %>` tags, easy learning curve, full JS logic support
- **Handlebars (hbs)** → HTML with `{{ }}` bindings, logic via helpers
- **Pug** → Indentation-based syntax, concise templates, slightly steeper curve
- Choose based on team preference, desired logic level, and readability needs

Template Engine Comparison

- **Syntax:** EJS (HTML + JS), Handlebars (HTML + helpers), Pug (whitespace driven)
- **Logic Support:** EJS & Pug allow inline JS; Handlebars prefers helpers
- **Use Cases:** EJS for dynamic content, Handlebars for simpler pages, Pug for minimal markup
- **Learning Curve:** EJS/Handlebars easy; Pug moderate but terse

EJS

```
<h1><%= title %></h1>
<ul>
  <% items.forEach(item => { %>
    <li><%= item %></li>
  <% }); %>
</ul>
```

Handlebars

```
<h1>{{title}}</h1>
<ul>
  {{#each items}}
    <li>{{this}}</li>
  {{/each}}
</ul>
```

Pug

```
h1= title
ul
  each item in items
    li= item
```

Layouts and Layout Workflow

- Layout files keep shared markup (header, nav, footer) in one place
- Individual views inject content via `<%- body %>` (EJS) or equivalent slots
- Pair with `express-ejs-layouts` or engine-specific layout helpers
- Keeps pages DRY and easier to maintain during redesigns

Why Use Layouts?

- Keeps your views **DRY** (Don't Repeat Yourself)
- Reuse common UI elements like header, footer, stylesheets & scripts
- Focus only on content in individual views

Common Layout Pattern (EJS)

1. Create `views/layout.ejs`
2. Include header/footer/styles
3. Use `<%- body %>` to inject page-specific content



Example: `layout.ejs`

```
<!DOCTYPE html>
<html>
  <head>
    <title><%= title %></title>
    <link rel="stylesheet" href="/stylesheets/style.css" />
  </head>
  <body>
    <header>
      <h1>My Web App</h1>
      <nav>
        <a href="/">Home</a> | <a href="/about">About</a>
      </nav>
    </header>
    <main>
      <%- body %>    <!-- your view here!!! -->
    </main>
    <footer>
      <p>© 2025 My Web App</p>
    </footer>
  </body>
</html>
```


Configure EJS to Use Layout

Install `express-ejs-layouts` :

```
npm install express-ejs-layouts
```

Update `app.js`

```
const expressLayouts = require('express-ejs-layouts');  
app.use(expressLayouts);  
app.set('layout', 'layout'); // Uses views/layout.ejs
```

Example View Using Layout:
`views/about.ejs`

```
<h2><%= title %></h2>  
<p>This is the About page content.</p>
```

Rendered inside the layout's `<%- body %>` tag

Express and Generator References

- [Templating Engines Comparison](#)
- [Express Generator Documentation](#)
- [Express Examples](#)

Part 3 - Databases

- RDBMSes
 - Differences between MySQL, PostgreSQL, MariaDB, MongoDB
- SQL language
- Setting up a DB connection Node <-> PostgreSQL on localhost
- Object Relational Management (ORMs)

Database Landscape

- **Relational:** PostgreSQL, MySQL, MariaDB emphasize structured schemas and SQL
- **Document / NoSQL:** MongoDB stores JSON-like docs with flexible schema
- **NewSQL / Cloud:** Services like CockroachDB blend SQL interfaces with distribution
- Select technology based on data shape, consistency needs, and team expertise

Relational Options at a Glance

- **PostgreSQL**: Standards-compliant, rich data types, powerful indexing and extensions
- **MySQL**: Ubiquitous, simple to start, strong community support
- **MariaDB**: Drop-in MySQL fork with performance enhancements
- We adopt **PostgreSQL** for its reliability, tooling, and Node.js ecosystem fit

Installing PostgreSQL on Ubuntu 24.04

- Update apt metadata, install server plus contrib extensions
- Verify the systemd service is active and accepting connections
- Use the default `postgres` superuser to open a `psql` session

```
sudo apt update
sudo apt install postgresql
sudo systemctl status postgresql
sudo -u postgres psql # use this whenever you need to connect directly to the db
```

Create Role and Database

- Create a dedicated login role for your app (never use the superuser)
- Make your application database owned by the new role
- Grant privileges so migrations and seeds can run cleanly

```
-- inside psql as the postgres superuser
CREATE ROLE webapp LOGIN PASSWORD 'change_me';
CREATE DATABASE webapp_db OWNER webapp;
GRANT ALL PRIVILEGES ON DATABASE webapp_db TO webapp;
\q
```

Configure Project Environment 🌱

- Store connection details in `.env` (keep it out of version control)
- `DATABASE_URL` plays nicely with ORMs and query builders (`pg` , Prisma, Sequelize)
- Reload your Node process after changing environment variables

```
PGHOST=localhost  
PGUSER=webapp  
PGPASSWORD=change_me  
PGDATABASE=webapp_db  
PGPORT=5432  
DATABASE_URL=postgresql://webapp:change_me@localhost:5432/webapp_db
```


Loading Environment Variables with dotenv 🌿

- Install `dotenv` to load `.env` contents into `process.env`
- Call `require('dotenv').config()` at the top of your entry file
- Access secrets via `process.env.PGHOST` (or `DATABASE_URL`) throughout the app

```
npm install dotenv pg
```

```
// app.js
require('dotenv').config();

const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

module.exports = pool;
```

Express Query from a Route

- Reuse a shared `Pool` instance to execute parameterized SQL inside handlers
- Log each row for debugging/analytics before returning a JSON response
- Bubble errors to Express error middleware with `next(err)`

```
// routes/people.js
const express = require('express');
const pool = require('../db');
const router = express.Router();

router.get('/', async (req, res, next) => {
  try {
    const { rows } = await pool.query(
      'SELECT id, first_name, email FROM people ORDER BY created_at DESC LIMIT 20'
    );

    rows.forEach(row => {
      console.log(`${row.id}: ${row.first_name} <${row.email} || 'n/a'>`);
    });

    res.json({ people: rows });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

SQL Building Blocks

- **Database:** container for schemas, tables, functions, views
- **Table:** rows + columns enforcing data types and constraints
- **Primary Key:** unique identifier for each row (`SERIAL` , `UUID` , etc.)
- **Foreign Key:** links rows across tables and enforces referential integrity
- **Constraint:** rules like `NOT NULL` , `UNIQUE` , `CHECK` , `DEFAULT`

Sample Table Layout

```
people
id | first_name | last_name | email | age
---|-----|-----|-----|----
1 | Jordan | Lee | jordan@example.com | 29
2 | Priya | Patel | priya@example.com | 32
```

- Columns map to data types (TEXT , INTEGER , BOOLEAN , TIMESTAMPTZ)
- Rows store individual records following the schema definition

SQL Statement Families

- DDL (Data Definition) → CREATE , ALTER , DROP
- DML (Data Manipulation) → INSERT , UPDATE , DELETE
- DQL (Query) → SELECT , WHERE , JOIN , ORDER BY
- DCL (Control) → GRANT , REVOKE for permissions

Creating Database Objects

```
-- connect to PostgreSQL and pick a database
\c webapp_db

-- create a table with constraints
CREATE TABLE people (
  id SERIAL PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

Inserting Data +

```
INSERT INTO people (first_name, last_name, email)
VALUES
  ('Jordan', 'Lee', 'jordan@example.com'),
  ('Priya', 'Patel', 'priya@example.com');
```

- Auto-incrementing columns like `SERIAL` generate IDs automatically
- Use parameterized queries in Node to avoid SQL injection vulnerabilities

Querying Data

```
SELECT * FROM people;
```

```
SELECT first_name, email  
FROM people  
WHERE created_at >= NOW() - INTERVAL '30 days'  
ORDER BY created_at DESC;
```

- `SELECT` chooses columns, `WHERE` filters rows, `ORDER BY` sorts output
- Combine with `LIMIT`, `OFFSET`, and aggregates (`COUNT`, `SUM`) for reports

Updating and Deleting Rows

```
UPDATE people  
SET email = 'jordan.lee@example.com'  
WHERE id = 1;
```

```
DELETE FROM people  
WHERE id = 2;
```

- Always double-check `WHERE` clauses before running destructive statements
- Wrap multi-step operations in transactions (`BEGIN; ... COMMIT;`) when needed

Foreign Keys and Relationships

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  person_id INTEGER NOT NULL REFERENCES people(id),  
  total_cents INTEGER NOT NULL,  
  placed_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
SELECT p.first_name, o.total_cents  
FROM people p  
JOIN orders o ON o.person_id = p.id;
```

- Foreign keys enforce valid relationships across tables
- Joins combine related tables for richer query results

SQL Workflow Recap

- Install PostgreSQL, create least-privilege roles, secure credentials
- Model entities with tables, keys, and constraints
- Use `INSERT` , `SELECT` , `UPDATE` , `DELETE` for CRUD workflows
- Apply joins and aggregates for analytics and reporting
- Track schema changes with migration scripts to keep teams aligned

Mapping Concepts to MySQL

- Replace `psql` with `mysql -u webapp -p` for CLI access and use `SHOW DATABASES;`
- Connection strings swap to `mysql://webapp:change_me@localhost:3306/webapp_db`
- Use `AUTO_INCREMENT` instead of `SERIAL` , `DATETIME` instead of `TIMESTAMPTZ`
- Swap `BOOLEAN` for `TINYINT(1)` or `BOOLEAN` (alias) when porting schemas
- `pg` client becomes `mysql2` or Prisma with a MySQL datasource URL

Part 4 - External APIs

- APIs and Proxying Requests
- Example calling Giphy
- Example calling Spotify
- Example calling Gemini

Backend API Gateway Strategy

- Expose Express routes as trusted proxies for third-party APIs
- Call external services server-side to dodge CORS issues cleanly
- Keep secrets on the server; clients only see `/api/...`
- Normalize responses so every consumer receives the same shape
- Centralize logging, caching, and rate protection in one layer

Proxy Request Lifecycle

- Browser calls `/api/gifs?q=cats`
- Express validates params and injects API keys
- Backend forwards request to provider (Giphy, Spotify, etc.)
- Response is sanitized, errors handled, then returned to client

```
// frontend/services/gifs.js
export async function searchGifs(query) {
  const res = await fetch(`/api/gifs?q=${encodeURIComponent(query)}`);
  if (!res.ok) throw new Error('Search failed');
  return res.json();
}
```

Express Proxy Setup Essentials

- Install helpers: `npm install node-fetch dotenv express-rate-limit`
- Load `.env` at startup and share JSON/body middleware
- Mount proxy routers under `/api` to prevent route clashes
- Apply rate limits or API-specific middleware per router

```
require('dotenv').config();
const express = require('express');
const rateLimit = require('express-rate-limit');

const app = express();
app.use(express.json());
app.use('/api', rateLimit({ windowMs: 60_000, max: 100 }));

app.use('/api/gifs', require('./routes/giphy'));
app.use('/api/spotify', require('./routes/spotify'));
app.use('/api/ai', require('./routes/ai'));
```


node-fetch Essentials 🐟

- Lightweight fetch implementation for Node versions without global `fetch`
- Fully compatible with Express: `await fetch(...)` inside route handlers
- Shares the browser API: `Response.json()`, streaming bodies, headers API
- Supports AbortController and timeouts to guard long-running proxy calls

```
const fetch = require('node-fetch');

app.get('/ping', async (req, res, next) => {
  try {
    const data = await fetch('https://httpbin.org/get').then(r => r.json());
    res.json({ origin: data.origin });
  } catch (err) {
    next(err);
  }
});
```

Proxy Security Checklist - Best Practices

- Keep `.env` keys out of Git; rotate them when students finish demos
- Rate-limit by IP and method; log bursts to detect abuse early
- Validate query/body params before hitting upstream APIs
- Strip or rewrite response headers that expose provider internals
- Prefer `node-fetch` timeouts and retries to avoid hung requests

Giphy Proxy Overview

- Register for a free key at developers.giphy.com
- Hit `/v1/gifs/search` with `q`, `limit`, `rating` controls
- Return id, title, and preview URL to keep responses light
- Store the key in `.env` —never ship it to the browser

```
GIPHY_API_KEY=replace_me
```

Express Route: Giphy Search

```
// routes/giphy.js
const router = require('express').Router();
const fetch = require('node-fetch');

router.get('/', async (req, res, next) => {
  const q = req.query.q?.trim();
  if (!q) return res.status(400).json({ error: 'Missing q' });

  try {
    const params = new URLSearchParams({
      api_key: process.env.GIPHY_API_KEY,
      q,
      limit: '10',
      rating: 'pg',
    });

    const { data } = await fetch(`https://api.giphy.com/v1/gifs/search?${params}`).then(r => r.json());

    res.json(
      data.map(({ id, title, images }) => ({ id, title, preview: images.fixed_height_small.url }))
    );
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Spotify API Fundamentals

- Register an app to receive client ID and secret
- Use the Client Credentials flow for server-to-server calls
- Cache tokens ($\approx$ 1 hour lifespan) and refresh automatically
- Start with search and audio feature endpoints for demos

```
SPOTIFY_CLIENT_ID=replace_me  
SPOTIFY_CLIENT_SECRET=replace_me  
SPOTIFY_API_BASE=https://api.spotify.com/v1
```

Spotify Token Helper

```
// lib/spotify-auth.js
const fetch = require('node-fetch');
let token;
let expiresAt = 0;

async function spotifyToken() {
  if (token && Date.now() < expiresAt) return token;

  const creds = Buffer.from(
    `${process.env.SPOTIFY_CLIENT_ID}:${process.env.SPOTIFY_CLIENT_SECRET}`
  ).toString('base64');

  const { access_token, expires_in } = await fetch('https://accounts.spotify.com/api/token', {
    method: 'POST',
    headers: {
      Authorization: `Basic ${creds}`,
      'Content-Type': 'application/x-www-form-urlencoded',
    },
    body: new URLSearchParams({ grant_type: 'client_credentials' }),
  }).then(res => res.json());

  token = access_token;
  expiresAt = Date.now() + expires_in * 1000 - 5_000;
  return token;
}

module.exports = { spotifyToken };
```

Express Route: Spotify Track Search - routes/spotify.js

```
const router = require('express').Router();
const fetch = require('node-fetch');
const { spotifyToken } = require('../lib/spotify-auth');

router.get('/tracks', async (req, res, next) => {
  const q = req.query.q?.trim();
  if (!q) return res.status(400).json({ error: 'Missing q' });

  try {
    const token = await spotifyToken();
    const params = new URLSearchParams({ type: 'track', q, limit: '8' });

    const { tracks } = await fetch(`${process.env.SPOTIFY_API_BASE}/search?${params}`, {
      headers: { Authorization: `Bearer ${token}` },
    }).then(res => res.json());

    res.json(
      tracks.items.map(({ id, name, artists, preview_url }) => ({
        id,
        name,
        artist: artists.map(a => a.name).join(', '),
        previewUrl: preview_url,
      }))
    );
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

AI API Options for Class Projects

- **Hugging Face Inference API:** REST access to Gemma, LLaMA, Mistral, etc.
- **Gemini API:** multimodal (text + images) with a generous free tier
- **Ollama:** run open models locally and expose an HTTP endpoint on campus
- Begin with text-only demos before tackling images or audio

```
HF_API_URL=https://api-inference.huggingface.co/models/google/gemma-2-2b-it  
HF_API_KEY=replace_me
```


Express Route: AI Text Generation - routes/ai.js

```
const router = require('express').Router();
const fetch = require('node-fetch');

router.post('/generate', async (req, res, next) => {
  const prompt = req.body.prompt?.trim();
  if (!prompt) return res.status(400).json({ error: 'Prompt is required' });

  try {
    const data = await fetch(process.env.HF_API_URL, {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${process.env.HF_API_KEY}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ inputs: prompt, parameters: { max_new_tokens: 120 } }),
    }).then(res => res.json());

    const [first] = Array.isArray(data) ? data : [data];
    res.json({ text: first?.generated_text ?? '' });
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Frontend Integration Tips 🧭

- Centralize API calls in a `services/` layer to reuse fetch logic
- Show loading + error states; upstream APIs throttle or fail at times
- Debounce search inputs calling Giphy/Spotify to reduce server load
- Document each proxy route (params, shape) so student teams stay aligned

Advanced Deployment Pathways

- Exercise routes with Postman or `curl` ; document expected inputs/outputs
- Script seed data and migrations so staging and prod stay in sync
- When internships call for it: Docker images plus PM2/Nginx process control
- For observability gigs: centralized logs (Logtail) and dashboards (Grafana)
- Always prepare a handoff bundle: API reference, env checklist, rollback plan